Homework 10

Solutions for Questions 1 and 2 will be available after Monday, November 2, 2026. No solution will be provided for Question 3.

Use the RLT package for Questions 1 and 2: `RLT` in R or `rlt` in Python. The node-size parameter is `nmin` in R and `min_samples_leaf` in Python; both stop splitting a node when it contains at most that many observations. The candidate-predictor parameter is `mtry` in both languages. Use ordinary random forests, with reinforcement turned off.

Question 1: Bias-variance trade-off simulation

Two important tuning parameters of random forests are node size and `mtry`. We have performed some preliminary analysis in class. The goal of this question is to perform a larger simulation study for these two tuning parameters.

Consider a regression model with five jointly normally distributed predictors. Each predictor has mean 0 and variance 1, and the correlation between any two different predictors is 0.3. The response is

$$Y = X_1 + X_3 + X_5 + \varepsilon, \qquad \varepsilon \sim N(0,1),$$

where the noise is independent of the predictors. Thus the true regression function is $f(\mathbf{x}) = x_1 + x_3 + x_5$.

a. Set up a simulation study with 100 repetitions. Generate 200 test predictor vectors once and keep them fixed. In each repetition, generate 200 new training observations and use the same training data for all tuning settings. Fit 500-tree forests, drawing 200 observations with replacement for each tree. Make the following two comparisons:

   - Fix `mtry` at 3 and vary node size over $1, 5, 10, 20, 30$.
   - Fix node size at 10 and vary `mtry` over $1, 2, 3, 4, 5$.

   Leave all other fitting parameters at their defaults. Record the predictions at each test point across repetitions. For each setting, estimate the squared bias and variance at each test point, then average them over the 200 test points. Compare predictions with $f(\mathbf{x})$ when estimating bias. Produce two figures, one for each tuning parameter, showing the averaged squared bias and variance. Set a random seed and report it.

b. Discuss your results in terms of the bias-variance trade-off:

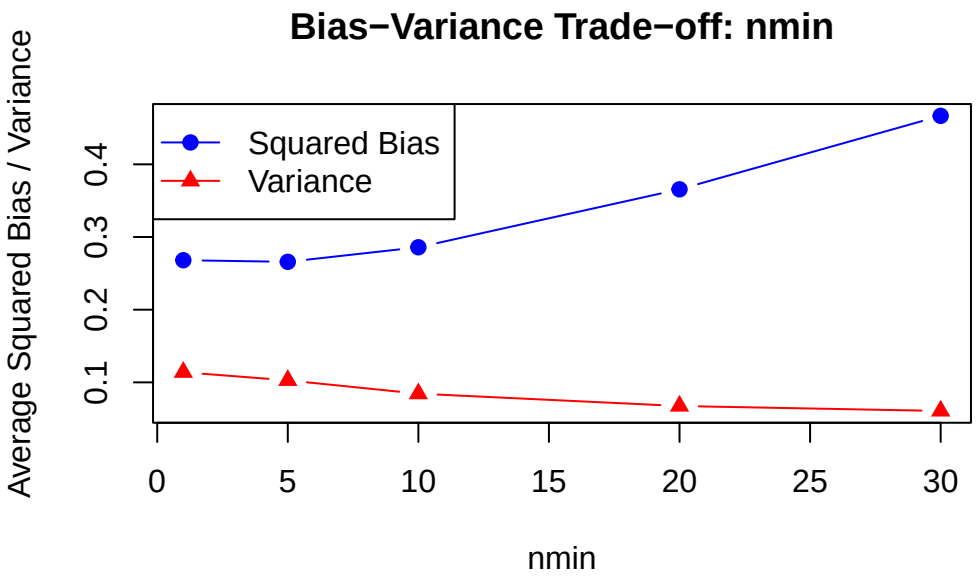
   - How does a large node size, such as 30, affect the fit in this example?
   - What are the benefits and drawbacks of using a large `mtry` value?
   - If each tree instead sampled 100 observations with replacement, would you expect the preferred node size to shift toward smaller or larger values? Explain your reasoning. You may use an additional simulation to help, but a particular pattern is not guaranteed.

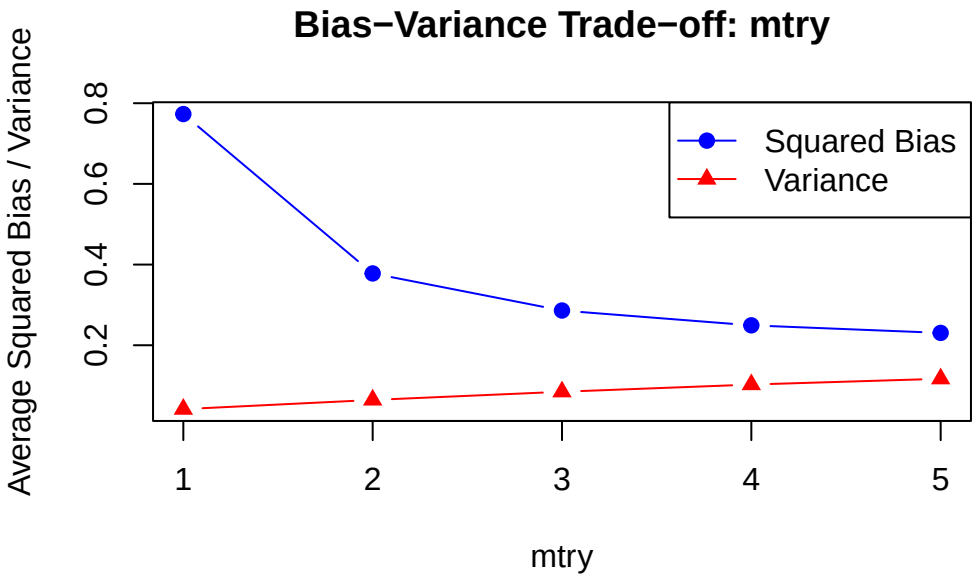
Answer

Part a

```
  nmin     Bias2   Variance
1    1 0.2680522 0.11396755
2    5 0.2658693 0.10275516
3   10 0.2858908 0.08445657
4   20 0.3656209 0.06745988
5   30 0.4667123 0.06066576
```

```
  mtry     Bias2   Variance
1    1 0.7731711 0.04160334
2    2 0.3780037 0.06445264
3    3 0.2861249 0.08472349
4    4 0.2493951 0.10254570
5    5 0.2306308 0.11695419
```





Part b

1. Effect of a large node size (nmin = 30)

A large node size produces shallower trees with fewer splits, making the model less flexible and more likely to underfit. In our simulation, increasing `nmin` from 1 to 30 increased squared bias from 0.268 to 0.467, while variance decreased from 0.114 to 0.061. Although larger nodes produce more stable predictions, the substantial increase in bias outweighs the reduction in variance. Therefore, `nmin = 30` results in underfitting in this example.

2. Benefits and drawbacks of a large mtry

A larger `mtry` allows more predictors to be considered at each split, increasing the likelihood of selecting informative variables (X_1, X_3, X_5). This reduces squared bias from 0.773 at `mtry = 1` to 0.231 at `mtry = 5`. However, larger `mtry` values make trees more similar, reducing the benefit of averaging and increasing variance from 0.042 to 0.117. In this simulation, the reduction in squared bias outweighs the increase in variance, so larger `mtry` values improve overall prediction performance.

3. Effect of reducing bootstrap sample size to 100

If each tree sampled only 100 observations with replacement instead of 200, I would expect the preferred node size to shift toward smaller values. With fewer observations available to each tree, a large `nmin` would restrict splitting more severely, producing simpler trees and potentially increasing bias. Smaller node sizes would allow trees to grow deeper and capture more of the underlying relationship. However, smaller bootstrap samples could also increase prediction variability, so this shift is not guaranteed and would need to be confirmed through simulation.

Question 2: Tuning random forests for MNIST

Use `mnist-first2000.csv.gz` from the combined ZIP. It contains 2,000 MNIST observations, with the digit label `Digit` and 784 pixel values.

- From the first 1,000 observations, keep digits 1, 6, and 7 as the training data. From the remaining 1,000 observations, keep the same digits as the test data.
- Fit PCA to the training pixels, centering but not scaling them. Use the first 20 principal components as predictors, and project the test observations using the same training means and loadings.

Complete the following tasks using RLT classification forests with 1,000 trees. For each tree, draw a bootstrap sample of the same size as the training set.

a. Fit a random forest with the default `mtry` and node-size settings. Report the out-of-bag (OOB) misclassification rate.

b. Construct a grid of `mtry` and node-size values to tune the model. Select the setting using OOB misclassification rate. Report the OOB error for each setting and the selected values. Calculate and present the selected forest's OOB permutation variable importance for the 20 principal components. Comment on your results.

c. Apply the selected model to the test data and report the misclassification rate and confusion matrix. Use the test labels only for this final assessment.

Answer

```
Training observations: 327

Test observations: 317


========== PART (a) ==========

Default mtry: 6

Default nmin: 5

Default OOB error: 0.03363914


========== PART (b) ==========
```

	mtry	nmin	OOB_Error
1	2	1	0.02446483
2	4	1	0.02752294
3	6	1	0.02752294
4	8	1	0.03058104
5	10	1	0.02752294
6	15	1	0.03058104
7	20	1	0.03669725
8	2	5	0.03058104
9	4	5	0.02446483
10	6	5	0.02752294
11	8	5	0.03363914
12	10	5	0.04281346
13	15	5	0.03363914
14	20	5	0.03669725
15	2	10	0.02446483
16	4	10	0.03669725
17	6	10	0.03363914
18	8	10	0.03363914
19	10	10	0.03363914
20	15	10	0.03669725
21	20	10	0.03669725
22	2	20	0.03363914
23	4	20	0.03363914
24	6	20	0.04281346
25	8	20	0.04281346
26	10	20	0.04281346
27	15	20	0.04281346
28	20	20	0.03975535

```
Selected tuning parameters:

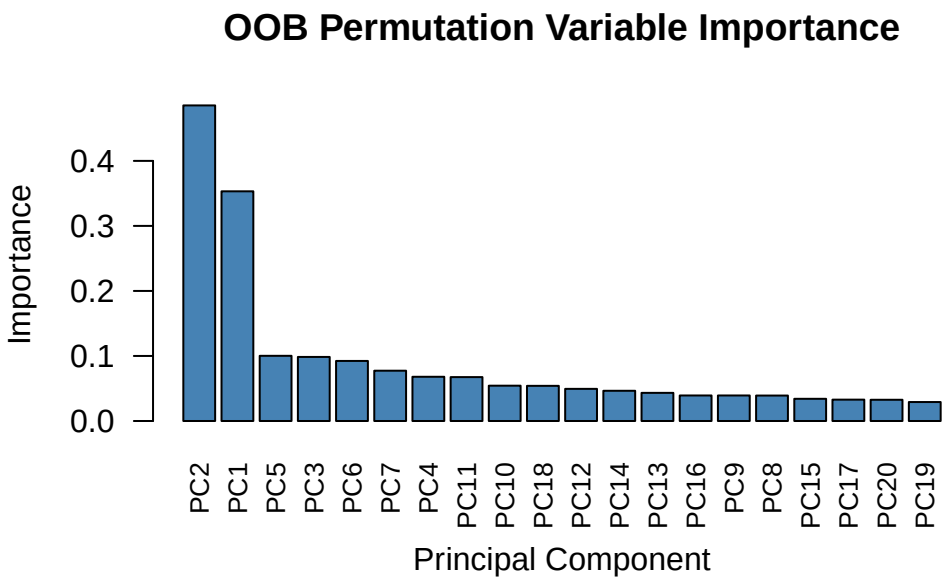
mtry = 2
```

```
nmin = 1

Minimum OOB error = 0.02446483


Permutation Variable Importance:

   PC Importance
  PC2 0.48529824
  PC1 0.35323869
  PC5 0.10021401
  PC3 0.09849126
  PC6 0.09234162
  PC7 0.07733022
  PC4 0.06800647
 PC11 0.06751781
 PC10 0.05427684
 PC18 0.05403762
 PC12 0.04939913
 PC14 0.04646308
 PC13 0.04324754
 PC16 0.03924669
  PC9 0.03923979
  PC8 0.03911202
 PC15 0.03412111
 PC17 0.03285626
 PC20 0.03265126
 PC19 0.02916305
```




```
========== PART (c) ==========

Test misclassification rate: 0.0126183
```

(a) Default Random Forest

The MNIST dataset was split into the first 1,000 observations for training and the remaining 1,000 for testing. After retaining only digits 1, 6, and 7, there were 327 training observations and 317 test observations.

PCA was performed on the training pixels with centering but no scaling. The first 20 principal components were used as predictors, and the test data were projected using the same PCA transformation.

An ordinary RLT classification forest with 1,000 trees was fitted using the default settings, `mtry = 6` and `nmin = 5`. Each tree used a bootstrap sample equal in size to the training set, and reinforcement was disabled.

The **OOB misclassification rate was 0.0336 (3.36%)**, corresponding to 11 misclassified observations out of 327.

(b) Tuning Random Forests and Variable Importance

A grid search was performed using seven `mtry` values (2, 4, 6, 8, 10, 15, 20) and four `nmin` values (1, 5, 10, 20), resulting in 28 tuning combinations.

The OOB misclassification rates ranged from 0.0245 to 0.0428, corresponding to 8–14 misclassified observations out of 327.

Three combinations achieved the minimum OOB error of **0.0245 (2.45%)**:

- `mtry = 2`, `nmin = 1`
- `mtry = 4`, `nmin = 5`
- `mtry = 2`, `nmin = 10`

The combination **`mtry = 2`, `nmin = 1`** was selected because it appeared first among the tied settings in the tuning grid.

The differences in OOB error across tuning settings were relatively small. The entire grid differed by at most six misclassified observations, and many configurations differed by only one or two observations. This suggests that classification performance was reasonably stable across the tuning parameters considered.

The selected forest's OOB permutation variable importance showed that **PC2 (0.485) and PC1 (0.353)** were the most important predictors. PC5, PC3, and PC6 followed with importance values around 0.09–0.10, while the remaining components had importance values of 0.08 or lower.

This suggests that the first few principal components contain much of the information useful for distinguishing digits 1, 6, and 7. However, PCA maximizes predictor variance rather than class separation, so a component's importance for classification is not necessarily determined by its explained variance.

(c) Test Set Evaluation

The selected random forest (`mtry = 2`, `nmin = 1`) was applied to the 317 test observations.

The resulting **test misclassification rate was 0.0126 (1.26%)**, corresponding to 4 incorrectly classified observations.

The confusion matrix was:

Actual / Predicted	1	6	7
1	103	0	1
6	1	105	0
7	0	2	105

Of the 104 actual digit 1 observations, 103 were correctly classified, with one misclassified as 7. Of the 106 digit 6 observations, 105 were correctly classified, with one misclassified as 1. Of the 107 digit 7 observations, 105 were correctly classified, with two misclassified as 6.

The test error of 1.26% was lower than the selected model's OOB error of 2.45%. This difference may be due to sampling variability and does not necessarily imply that the model will consistently perform better on test data than its OOB estimate suggests.

Overall, the tuned random forest achieved strong classification performance on the three selected MNIST digits, with relatively small differences across the tuning configurations.

Question 3: Planning a random-forest analysis skill

In earlier homework, some students wrote long skill files with detailed instructions, while others wrote short skills emphasizing their specific requirements or references. These approaches may perform differently depending on the AI and the task. A capable AI may already know standard ways to perform an analysis, so some instructions may be redundant, add little value, or sometimes make the analysis worse. It may be more useful to specify your personal choices for the analysis. An AI with more limited knowledge may benefit from detailed instructions that remind it of the steps to perform.

a. Write one paragraph describing how you would write a skill that directs an AI to perform a random-forest analysis and produce a report for a future dataset. Discuss how you would balance standard analysis instructions with your personal preferences, considering the AI's capabilities.

b. Provide **10 instructions** you would consider including in the skill, **ordered from the most basic instructions to the more personal preferences**. Your list may cover computation, tuning, tables, figures, interpretation, presentation, or other aspects of a random-forest report.

You only need to discuss how the skill should be written. You are not required to write a complete skill file or test it.

Answer

Part a

I would write a concise but structured skill that guides an AI through a random-forest analysis without unnecessarily explaining standard procedures it already understands. For a capable AI, I would focus more on specifying my preferences, such as using appropriate evaluation metrics, tuning parameters systematically, ensuring reproducibility, and producing a clear report. However, for a less capable AI, I would include more detailed instructions on data preparation, model fitting, tuning, and interpretation to reduce errors. I would also emphasize avoiding data leakage, using appropriate validation methods, and distinguishing training performance from test performance. My goal would be to create a reusable skill that provides enough guidance for a reliable analysis while allowing the AI flexibility to adapt its approach to different datasets.

Part b

1. **Data preparation:** Inspect the dataset, identify the response and predictors, handle missing values, and determine whether the task is regression or classification.
2. **Model fitting:** Fit a random forest using an appropriate package and clearly state the model settings, including the number of trees, `mtry`, and node size.
3. **Reproducibility:** Set a random seed and report it along with the software package and relevant model parameters.
4. **Model evaluation:** Evaluate performance using suitable metrics, such as misclassification rate for classification or mean squared error for regression.
5. **Hyperparameter tuning:** Compare multiple values of `mtry` and node size using out-of-bag error or cross-validation, and select the model based on a clearly stated criterion.
6. **Data leakage prevention:** Keep test data separate from model selection and perform any preprocessing using training data only.
7. **Variable importance:** Calculate permutation variable importance and identify the predictors that contribute most to the model's predictions.
8. **Results presentation:** Present tuning results in a concise table and include figures showing how model performance changes across tuning parameters.
9. **Interpretation:** Discuss the effects of tuning parameters on model complexity, bias, variance, and prediction performance rather than reporting numerical results alone.
10. **Report style:** Produce a concise, well-organized report with clearly labeled tables and figures, explain important findings in plain language, and avoid unnecessary technical details or repetitive explanations.